Sequence Diagrams

Interaction Diagrams

A series of diagrams describing the *dynamic behavior* of an object-oriented system.

- A set of messages exchanged among a set of objects within a context to accomplish a purpose.

Often used to model the way a use case is realized through a sequence of messages between objects.

Interaction Diagrams (Cont.)

The purpose of Interaction diagrams is to:

- Model interactions between objects
- Assist in understanding how a system (a use case) actually works
- Verify that a use case description can be supported by the existing classes
- Identify responsibilities/operations and assign them to classes

Interaction Diagrams (Cont.)

UML

- Collaboration Diagrams
  - Emphasizes structural relations between objects
- Sequence Diagram
  - The subject of this tutorial

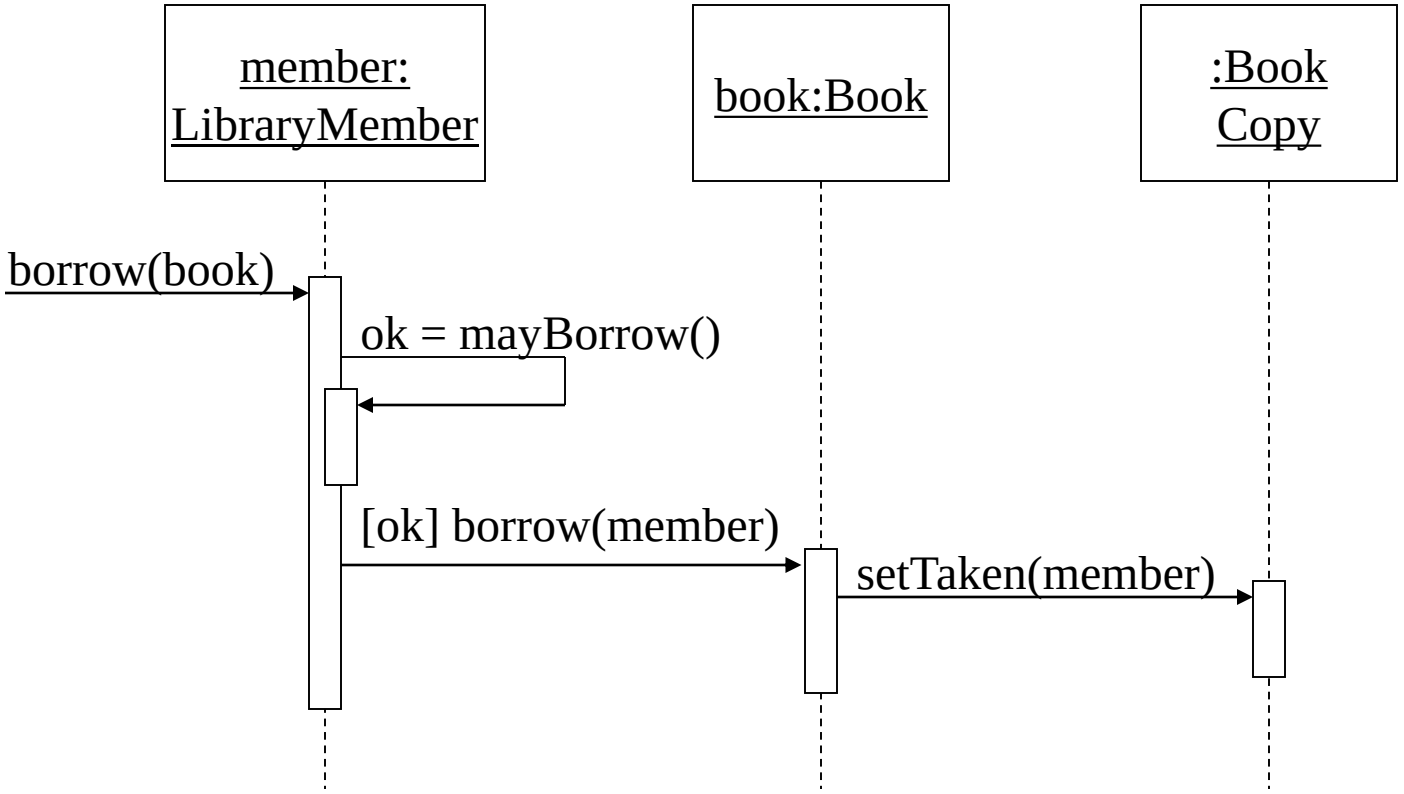
A First Look at Sequence Diagrams

Illustrates how objects interacts with each other.

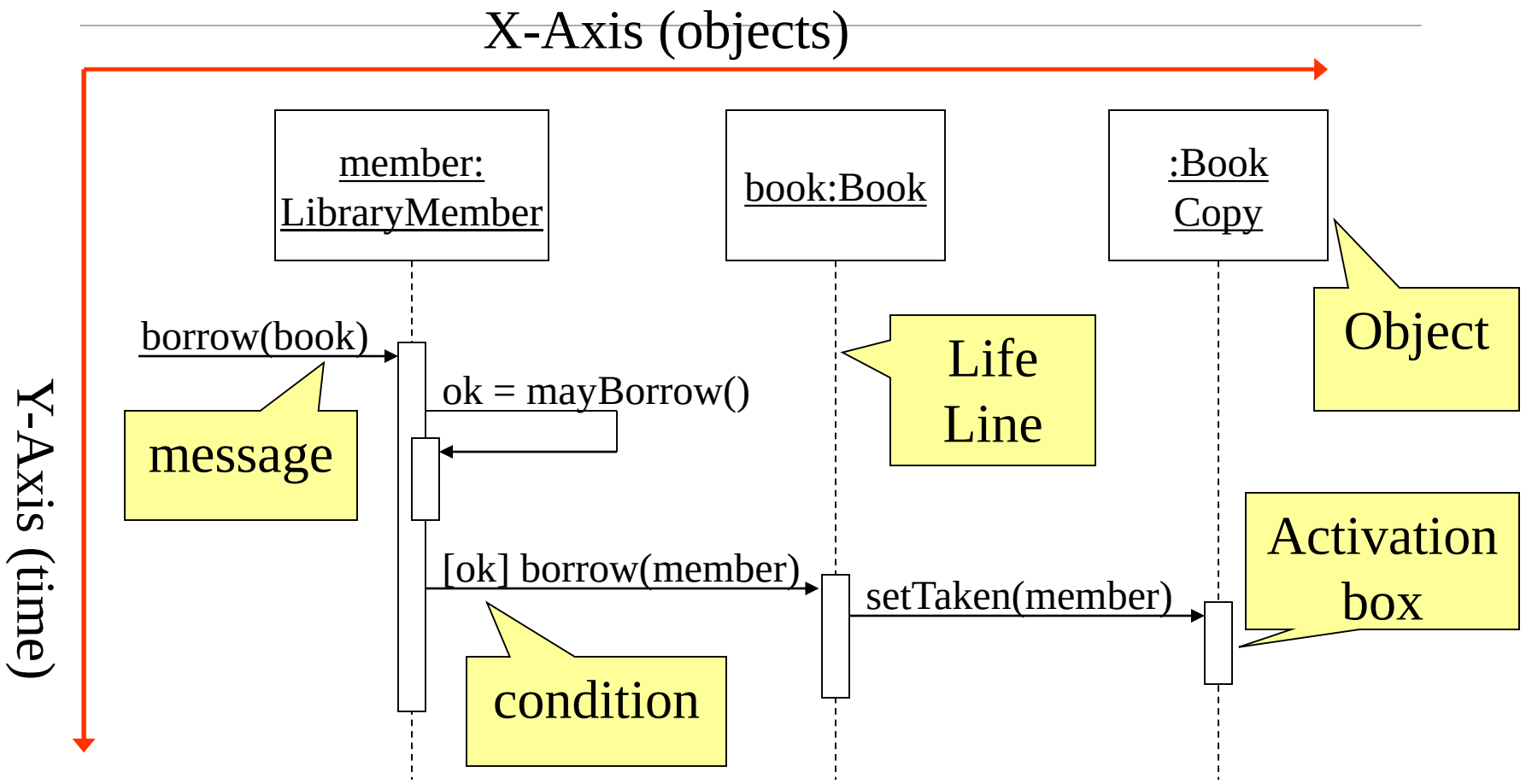
Emphasizes time ordering of messages.

Can model simple sequential flow, branching, iteration, recursion and concurrency.

A Sequence Diagram



A Sequence Diagram

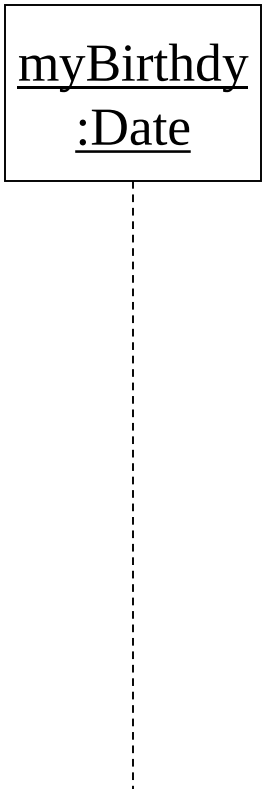


Object

Object naming:

- syntax: *[instanceName][:className]*
- Name classes consistently with your class diagram (same classes).
- Include instance names when objects are referred to in messages or when several objects of the same type exist in the diagram.

The *Life-Line* represents the object's life during the interaction



Messages

An interaction between two objects is performed as a message sent from one object to another (simple operation call, Signaling)

If object obj_1 sends a message to another object obj_2 some link must exist between those two objects (dependency, same objects)

Messages (Cont.)

A message is represented by an arrow between the life lines of two objects.

- Self calls are also allowed
- The time required by the receiver object to process the message is denoted by an *activation-box.*

A message is labeled at minimum with the message name.

- Arguments and control information (conditions, iteration) may be included.

Return Values



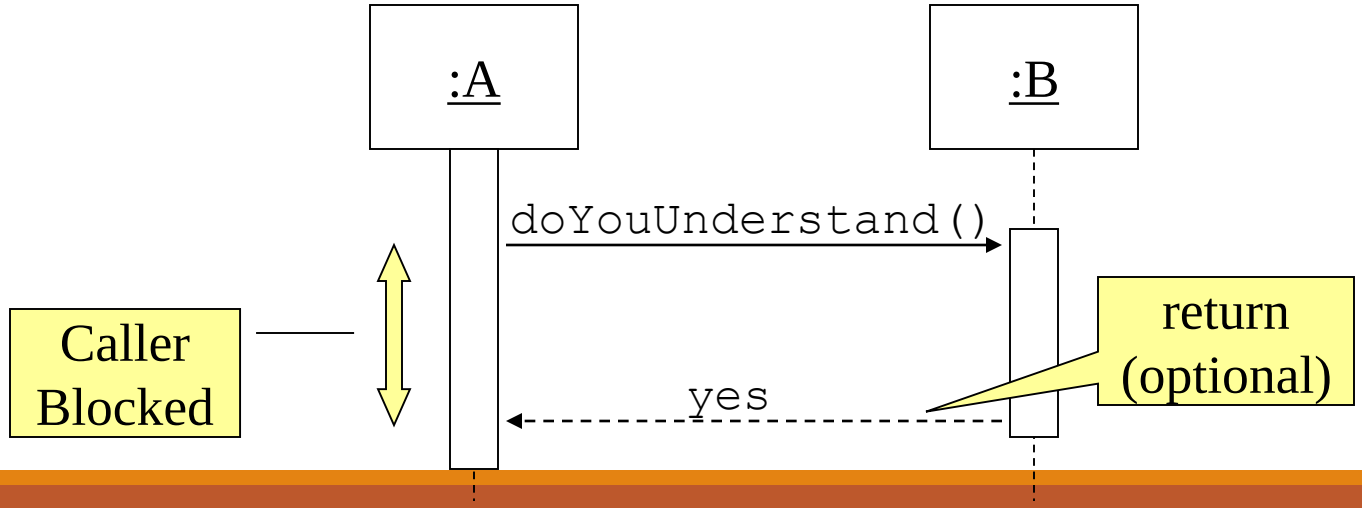
Optionally indicated using a dashed arrow with a label indicating the return value.

- **Don't model a return value when it is obvious what is being returned, e.g. getTotal()**
- **Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.**
- **Prefer modeling return values as part of a method invocation, e.g. `ok = isValid()`**

Synchronous Messages

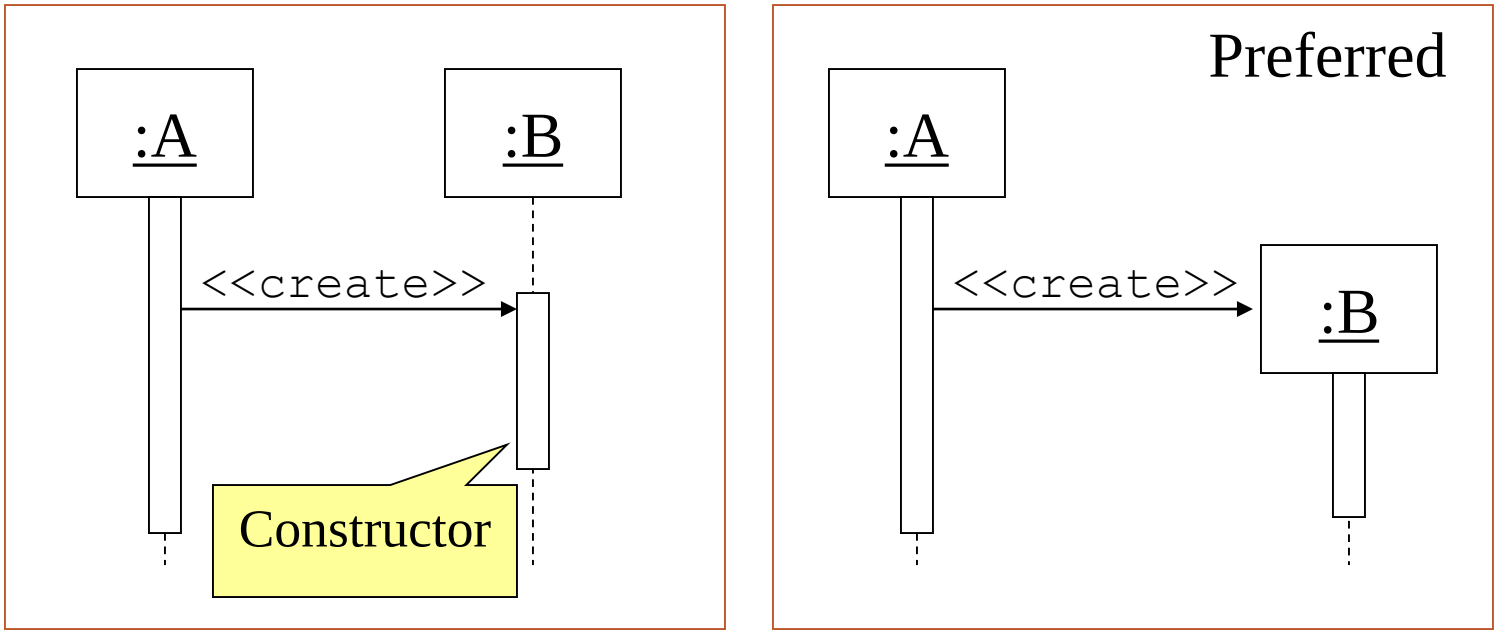
Nested flow of control, typically implemented as an operation call.

- The routine that handles the message is completed before the caller resumes execution.



Object Creation

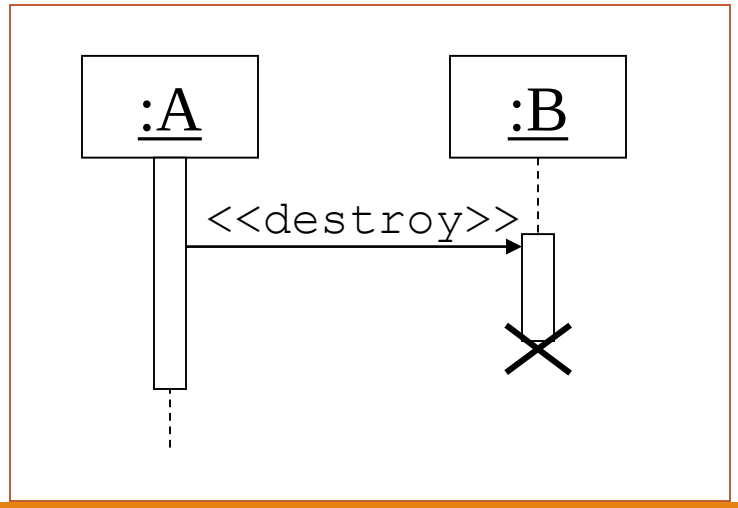
An object may create another object via a **<<create>>** message.



Object Destruction

An object may destroy another object via a **<<destroy>>** message.

- An object may destroy itself.
- Avoid modeling object destruction unless memory management is critical.



Control information

Condition

- syntax: ‘[‘ expression ’]’ message-label
- The message is sent only if the condition is true
- example:

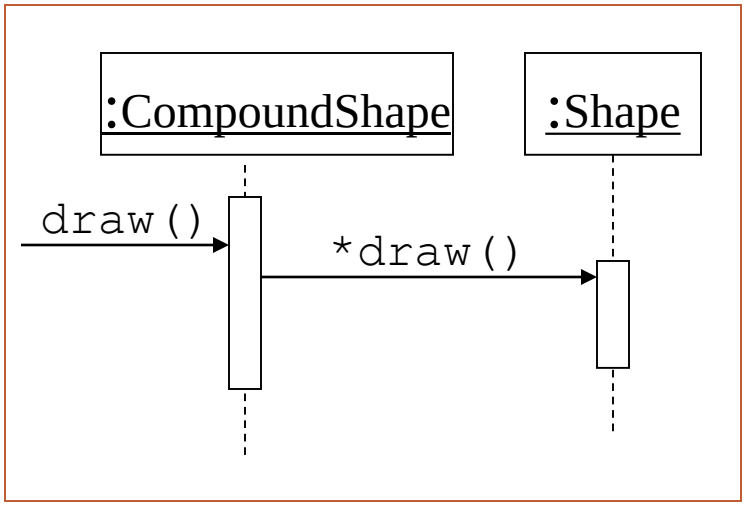


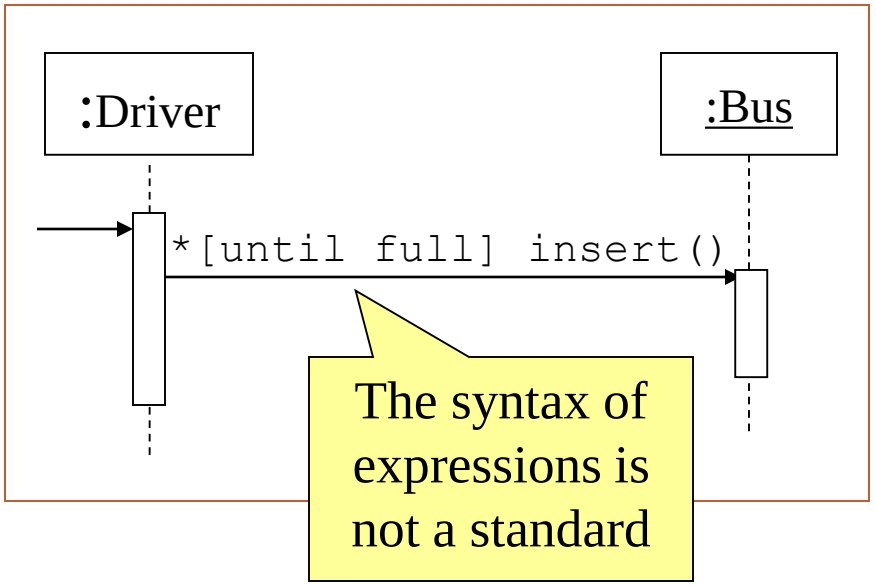
Iteration

- syntax: * [‘[‘ expression ‘]’] message-label
- The message is sent many times to possibly multiple receiver objects.

Control Information (Cont.)

Iteration examples:



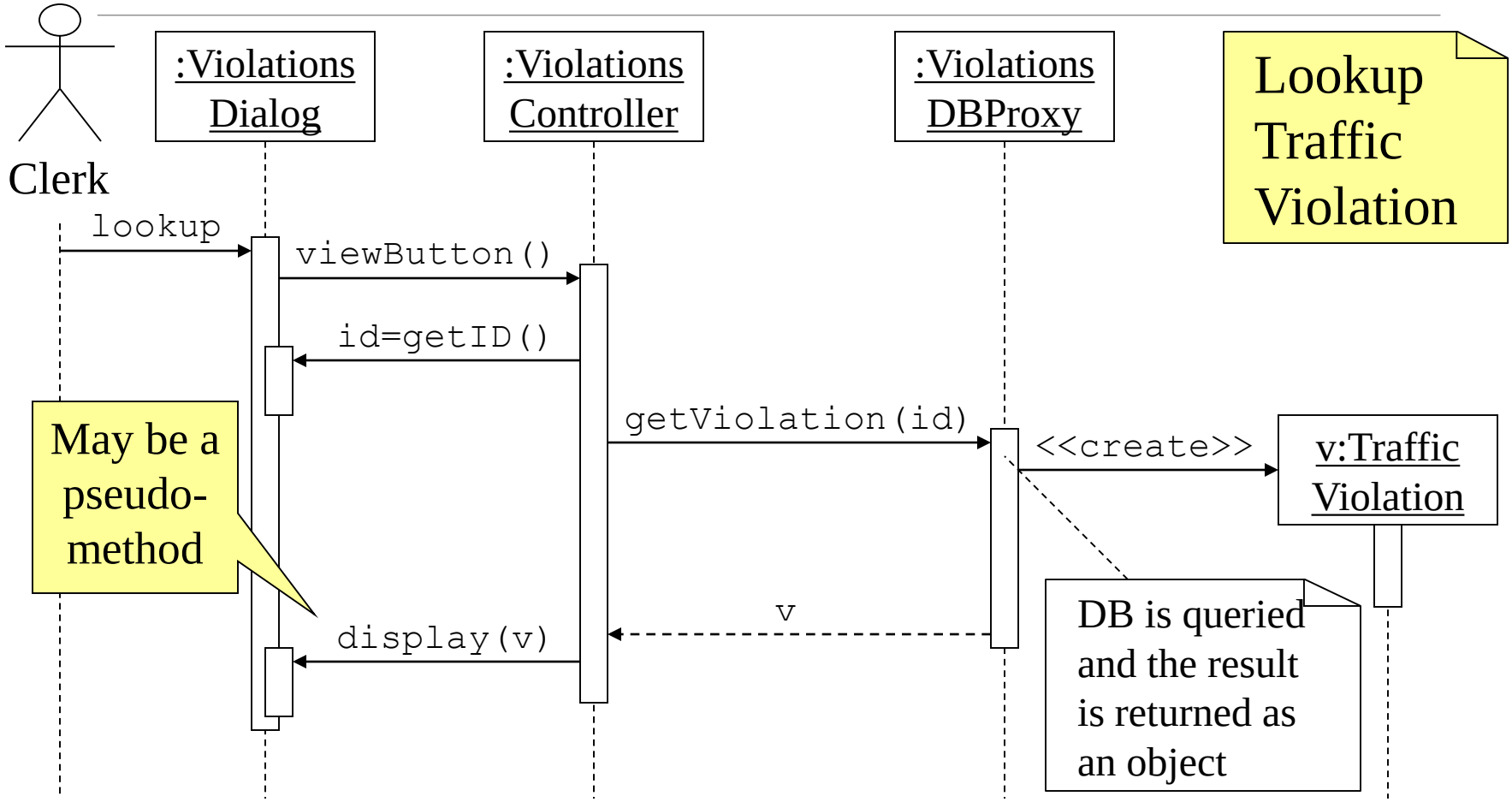


Control Information (Cont.)

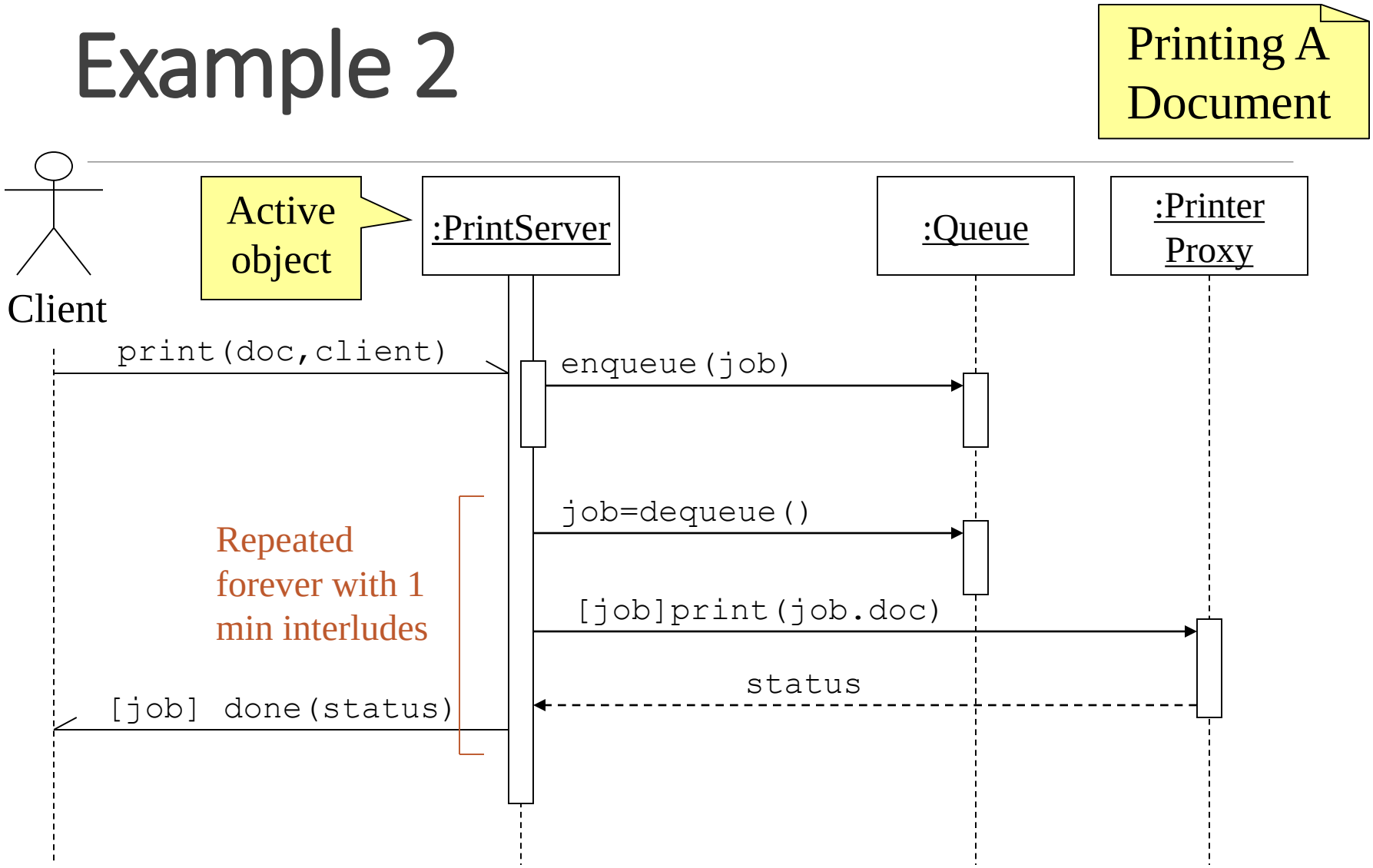
The control mechanisms of sequence diagrams suffice only for modeling simple alternatives.

- Consider drawing several diagrams for modeling complex scenarios.
- Don't use sequence diagrams for detailed modeling of algorithms (this is better done using *activity diagrams, pseudo-code* or *state-charts*).

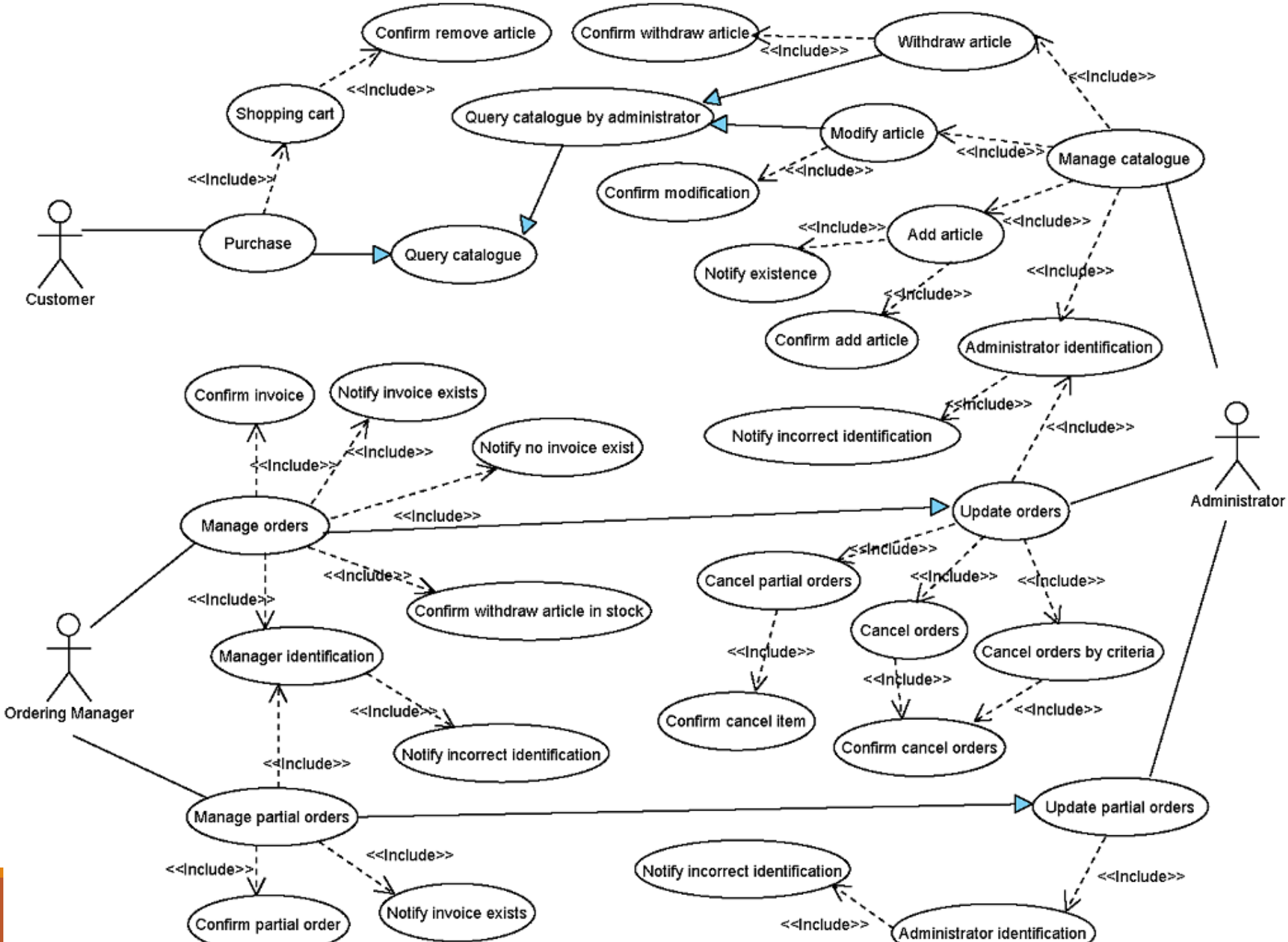
Example 1


Clerk
:Violations Dialog
:Violations Controller
:Violations DBProxy
Lookup Traffic Violation
lookup
viewButton()
id=getID()
getViolation(id)
<<create>>
v:Traffic Violation
May be a pseudo-method
v
display(v)
DB is queried and the result is returned as an object

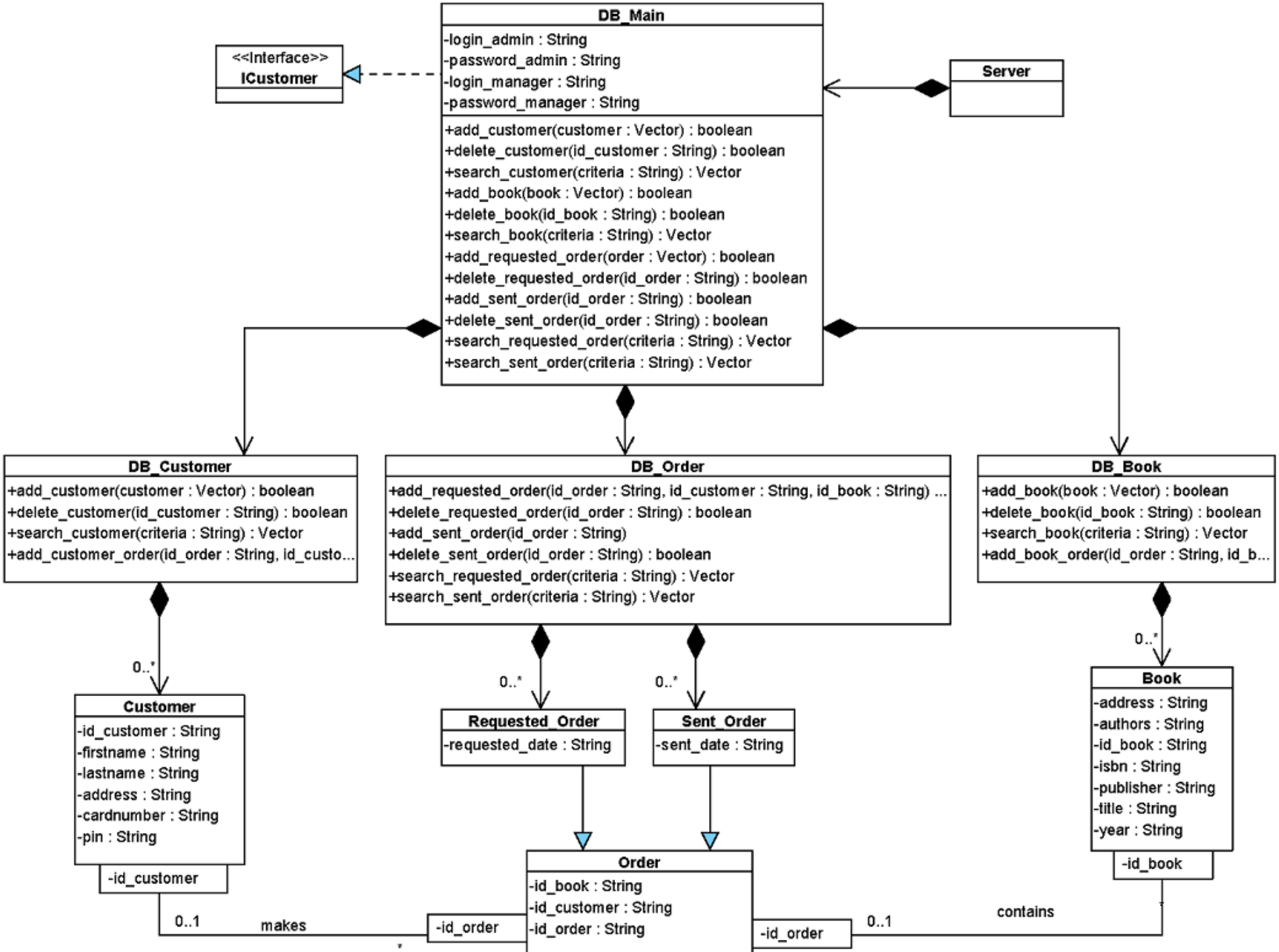
Example 2


Printing A Document
Client
Active object
:PrintServer
:Queue
:Printer Proxy
print(doc,client)
enqueue(job)
job=dequeue()
Repeated forever with 1 min interludes
[job]print(job.doc)
status
[job] done(status)

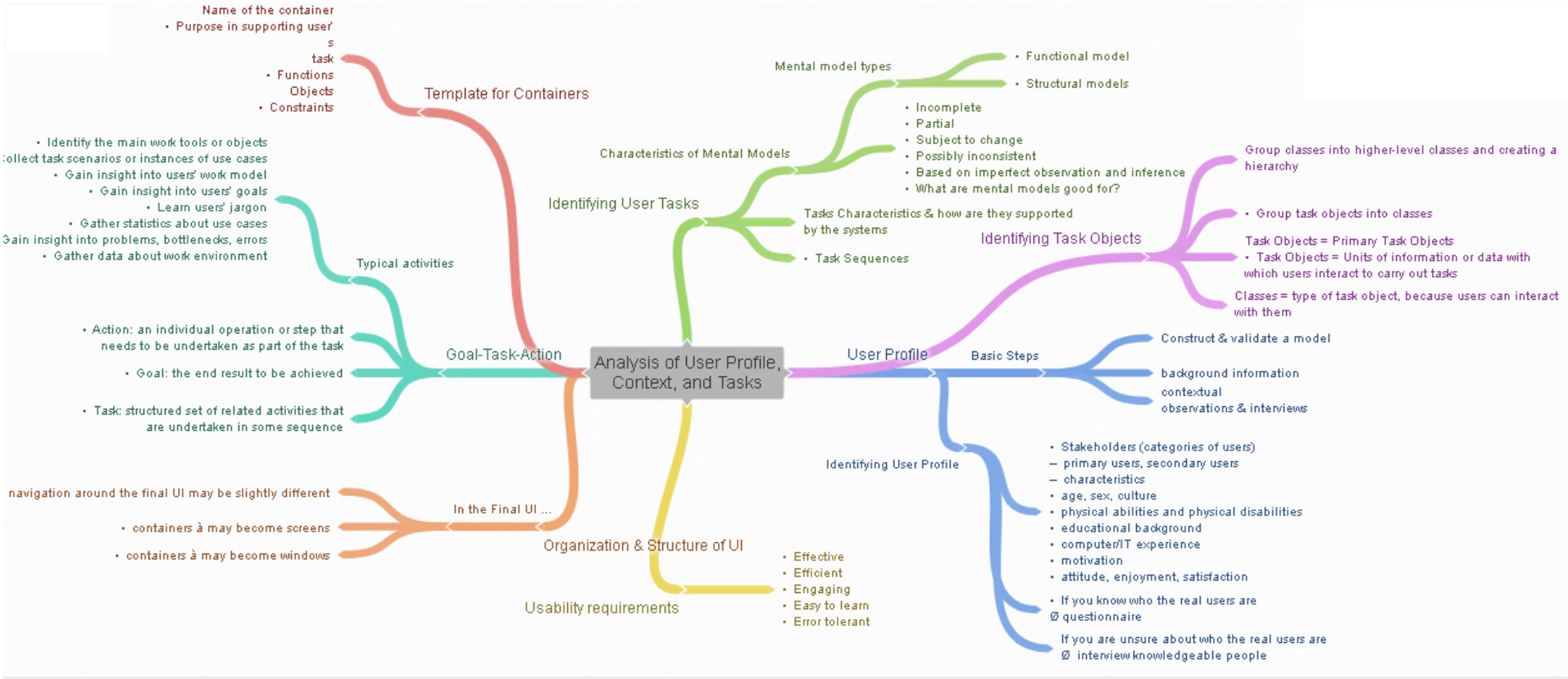
Use-Case Diagram Example


Confirm remove article
Confirm withdraw article
Withdraw article
<<Include>>
Shopping cart
Query catalogue by administrator
Modify article
Manage catalogue
Confirm modification
Purchase
Query catalogue
Customer
Add article
Notify existence
Confirm add article
Administrator identification
Confirm invoice
Notify invoice exists
Notify no invoice exist
Notify incorrect identification
Administrator
Manage orders
Update orders
Cancel partial orders
Cancel orders
Cancel orders by criteria
Confirm withdraw article in stock
Manager identification
Ordering Manager
Confirm cancel item
Confirm cancel orders
Notify incorrect identification
Manage partial orders
Update partial orders
Confirm partial order
Notify invoice exists
Notify incorrect identification
Administrator identification

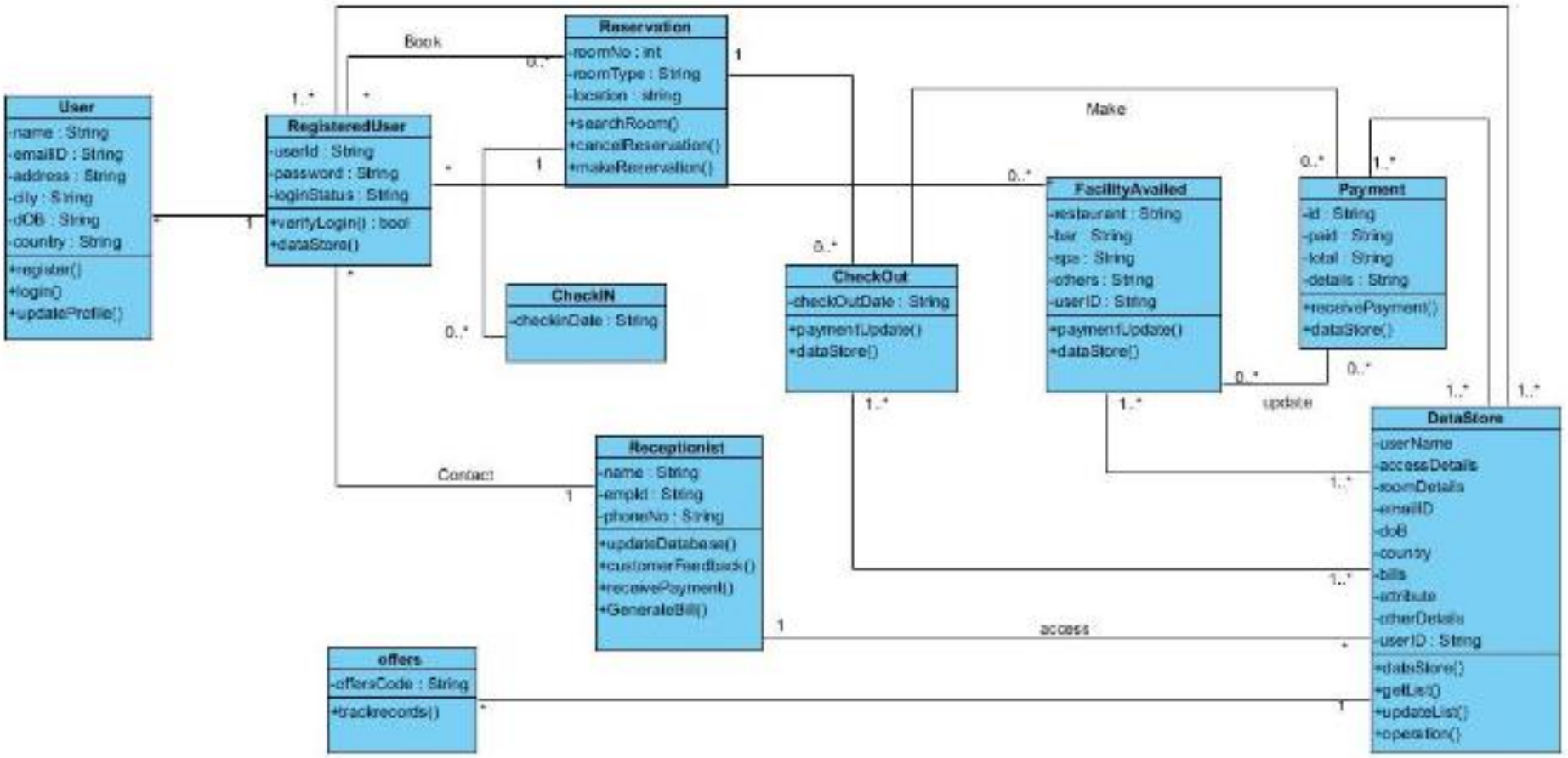
Database Diagram Example


DB_Main
-login_admin : String
-password_admin : String
-login_manager : String
-password_manager : String
+add_customer(customer : Vector) : boolean
+delete_customer(id_customer : String) : boolean
+search_customer(criteria : String) : Vector
+add_book(book : Vector) : boolean
+delete_book(id_book : String) : boolean
+search_book(criteria : String) : Vector
+add_requested_order(order : Vector) : boolean
+delete_requested_order(id_order : String) : boolean
+add_sent_order(id_order : String) : boolean
+delete_sent_order(id_order : String) : boolean
+search_requested_order(criteria : String) : Vector
+search_sent_order(criteria : String) : Vector
<<Interface>>
ICustomer
Server
DB_Customer
+add_customer(customer : Vector) : boolean
+delete_customer(id_customer : String) : boolean
+search_customer(criteria : String) : Vector
+add_customer_order(id_order : String, id_custo...
DB_Order
+add_requested_order(id_order : String, id_customer : String, id_book : String) ...
+delete_requested_order(id_order : String) : boolean
+add_sent_order(id_order : String)
+delete_sent_order(id_order : String) : boolean
+search_requested_order(criteria : String) : Vector
+search_sent_order(criteria : String) : Vector
DB_Book
+add_book(book : Vector) : boolean
+delete_book(id_book : String) : boolean
+search_book(criteria : String) : Vector
+add_book_order(id_order : String, id_b...
0..*
Customer
-id_customer : String
-firstname : String
-lastname : String
-address : String
-cardnumber : String
-pin : String
-id_customer
0..1
makes
0..*
-id_order
0..*
Requested_Order
-requested_date : String
0..*
Sent_Order
-sent_date : String
Order
-id_book : String
-id_customer : String
-id_order : String
-id_order
0..1
contains
0..*
Book
-address : String
-authors : String
-id_book : String
-isbn : String
-publisher : String
-title : String
-year : String
-id_book

User Profile Tasks


Analysis of User Profile, Context, and Tasks
Template for Containers
Name of the container
• Purpose in supporting user's task
• Functions
Objects
• Constraints
Goal-Task-Action
Typical activities
• Identify the main work tools or objects
Collect task scenarios or instances of use cases
• Gain insight into users' work model
• Gain insight into users' goals
• Learn users' jargon
• Gather statistics about use cases
Gain insight into problems, bottlenecks, errors
• Gather data about work environment
• Action: an individual operation or step that needs to be undertaken as part of the task
• Goal: the end result to be achieved
• Task: structured set of related activities that are undertaken in some sequence
In the Final UI ...
navigation around the final UI may be slightly different
• containers à may become screens
• containers à may become windows
Identifying User Tasks
Characteristics of Mental Models
Mental model types
• Functional model
• Structural models
• Incomplete
• Partial
• Subject to change
• Possibly inconsistent
• Based on imperfect observation and inference
• What are mental models good for?
Tasks Characteristics & how are they supported by the systems
• Task Sequences
Organization & Structure of UI
Usability requirements
• Effective
• Efficient
• Engaging
• Easy to learn
• Error tolerant
User Profile
Identifying Task Objects
Group classes into higher-level classes and creating a hierarchy
• Group task objects into classes
Task Objects = Primary Task Objects
• Task Objects = Units of information or data with which users interact to carry out tasks
Classes = type of task object, because users can interact with them
Basic Steps
Construct & validate a model
background information
contextual observations & interviews
Identifying User Profile
• Stakeholders (categories of users)
– primary users, secondary users
– characteristics
• age, sex, culture
• physical abilities and physical disabilities
• educational background
• computer/IT experience
• motivation
• attitude, enjoyment, satisfaction
• If you know who the real users are
Ø questionnaire
If you are unsure about who the real users are
Ø interview knowledgeable people

UML Diagram for Data Storage


User
-name : String
-emailID : String
-address : String
-city : String
-dOB : String
-country : String
+register()
+login()
+updateProfile()
RegisteredUser
-userId : String
-password : String
-loginStatus : String
+verifyLogin() : bool
+dataStore()
Reservation
-roomNo : int
-roomType : String
-location : string
+searchRoom()
+cancelReservation()
+makeReservation()
Book
CheckIN
-checkinDate : String
CheckOut
-checkOutDate : String
+paymentUpdate()
+dataStore()
Make
FacilityAvailed
-restaurant : String
-bar : String
-spa : String
-others : String
-userID : String
+paymentUpdate()
+dataStore()
Payment
-id : String
-paid : String
-total : String
-details : String
+receivePayment()
+dataStore()
update
Contact
Receptionist
-name : String
-empId : String
-phoneNo : String
+updateDatabase()
+customerFeedback()
+receivePayment()
+GenerateBill()
access
DataStore
-userName
-accessDetails
-roomDetails
-emailID
-dob
-country
-bills
-attribute
-otherDetails
-userID : String
+dataStore()
+getList()
+updateList()
+operation()
offers
-offersCode : String
+trackrecords()